

1	Intelligent Agents
2	Introduction to Machine Learning
3	Regularization
4	Kernel & Feature Transformation
5	Neural Networks
6	Convolutional Neural Networks (CNNs)
7	Unsupervised Learning with NN
8	Uncategorized

1 Intelligent Agents

1.1 Characterizing the Environment

- Fully observable** - (vs. Partially) Agent's sensors can access complete state of env. all the time
- Deterministic** - (vs. Stochastic) Next state of env. is determined by **current state and action executed by agent**
 - Strategic** - If env. is deterministic except for actions of other agents
- Episodic** - (vs. Sequential) Env.'s experience is divided into atomic **episodes**, where each episode includes perceiving and an action, and **action depends on episode itself**
- Static** - (vs. Dynamic) Env. is unchanged while agent is deciding
 - Semi** - Time does not affect env., but affects performance score
- Discrete** - (vs. Continuous) Discrete num. of percepts and actions
- Single Agent** - (vs. Multi-agent) Agent operating by itself in an env.

2 Introduction to Machine Learning

- A machine learns if it improves performance P on task T based on experience E. Where T must be fixed, P must be measurable, E must exist

2.0.1 Types of Feedback

- Supervised** - Correct answer given for each example
 - Regression** - Predict results within continuous output
 - Classification** - Predict results in discrete output
- Unsupervised** - No answers given
- Weakly supervised** - Answer given, but not precise
- Reinforcement** - Occasional rewards given

2.1 Decision Trees

- DT can express any function of input attributes, if data is consistent
- Goal: Make DT **compact**. How?

2.1.1 Information Theory

- Idea: Choose attribute that splits examples into subsets that are ideally 'all positive' or 'all negative'
- Entropy** - Measure of randomness in set of data

2.2 Performance Measurement

- Correctness** - Correct if $\hat{y} = y$
- Accuracy** - $\frac{1}{m} \sum_{j=1}^m (\hat{y}_j = y_j)$

- Confusion Matrix:

	Actual Label	
	+ve	-ve
+ve	TP True Positive	FP False Positive
-ve	FN False Negative	TN True Negative

- Type I Error: FP, Type II Error: FN
- FP Rate = $\frac{FP}{FP+TN}$ TP Rate = $\frac{TP}{TP+FN}$

2.3 Pruning

- Motivation: DT overfits to training set, but performs poorly on test set
- Occam's Razor: Simple hypothesis preferred
- Pruning** - Ignores outliers, which reduces overfitting
 - Idea: Go with the majority of T/Fs
 - E.g. Min-sample, Max-depth

3 Regularization

3.1 Overfitting

- Model learns training data too well $\Rightarrow$ high training accuracy, poor generalization
- Caused by: model complexity > ground truth complexity

3.2 Bias-Variance Tradeoff

- High bias models are too simple and cannot capture patterns well
- High variance models are too complex and are overfitted to training data
- We use regularization on high variance models to get the model to learn ground truth patterns rather than memorize the data
- a good model has balanced bias and variance in a way that fits the task

3.3 L_p -norm

Vector size measure ($p \geq 1$):

$$\|w\|_p = \left(\sum_{j=0}^d |w_j|^p \right)^{1/p}$$

- L_1 : $\|w\|_1 = \sum_j |w_j|$ (Manhattan)
- L_2 : $\|w\|_2 = \sqrt{\sum_j w_j^2}$ (Euclidean)
- L_∞ : $\|w\|_\infty = \max_j |w_j|$ (Max)

3.4 Regularization

Adds penalty to prevent model complexity: $J_{\text{reg}}(w) = J(w) + \lambda R(w)$

- $\lambda \geq 0$: regularization strength
- $R(w)$: penalty function (norm-based)
- Observation: overfitting $\Leftrightarrow$ large weight magnitudes

3.4.1 Common Regularizers

- Lasso: $J(w) + \lambda \|w\|_1$ (sparse solutions)
- Ridge: $J(w) + \lambda \|w\|_2$ (weight shrinkage)
- Elastic Net: $J(w) + \lambda_1 \|w\|_1 + \lambda_2 \|w\|_2$

3.5 Ridge Regression (L_2 Regularization)

Closed-form: $w = (X^T X + \lambda I)^{-1} X^T y$

- Invertibility: $X^T X + \lambda I$ invertible $\forall \lambda > 0$
- Unique solution exists
- Shrinks weights uniformly, none become zero

3.6 Lasso Regression (L_1 Regularization)

- Induces sparsity: some weights exactly zero
- Threshold rule: contribution to MSE reduction < penalty $\lambda \Rightarrow$ weight set to 0
- Feature selection automatic

4 Kernel & Feature Transformation

4.1 The Kernel Function:

$$K(u, v) = u \cdot v \quad (\text{dot product of two input vectors})$$

where $u, v \in \mathbb{R}^d$ are two input points, returns a scalar. Kernels = similarity function in feature space, not distance.

4.2 The Feature Map:

$$\phi(x): \mathbb{R}^d \rightarrow \mathbb{R}^M \quad (\text{transforms input into feature space})$$

For the linear kernel: $\phi(x) = x$ (identity transformation)

4.3 Fundamental Kernel-Feature Relationship:

$$K(u, v) = \phi(u) \cdot \phi(v)$$

The kernel equals the dot product of transformed feature vectors.

4.3.1 examples

Kernel Formulas: $K_{\text{new}}(u, v) = 4 \times K(u, v) = 4(u \cdot v) = (2u \cdot 2v)$ Scaling factor: 4, OR

$K(u, v) = K(u, v) = \log(u) \cdot \log(v) = \log(u)^T \cdot \log(v) = \log(u_1)^T \log(v_1) \cdot \log(u_2)^T \log(v_2)$

The New Feature Map (Unknown):

$$\phi_{\text{new}}(x) = 2x$$

We need to find the feature map corresponding to K_{new} .

4.4 The Prediction Formula (Kernel Methods)

$$h_a(x) = \sum_{i=1}^N a_i K(x^{(i)}, x)$$

where:

- $x^{(i)} = i$ -th training point
- a_i = learned weight for training point i
- $K(x^{(i)}, x)$ = kernel between training point i and new point x
- N = total number of training points

4.5 Feature Transformation

$$x \in \mathbb{R}^d \xrightarrow{\phi} \phi(x) \in \mathbb{R}^M \quad (M \geq d)$$

Polynomial degree k : $\binom{d+k}{k}$ features

- 2 features, degree 2: $\binom{2+2}{2} = 6$ features

4.6 Linear Combination of Vectors

$$v = \sum_{i=1}^n a_i x^{(i)}, \quad a_i \in \mathbb{R}$$

Geometric interpretation: v lies in span of $\{x^{(1)}, \dots, x^{(n)}\}$

4.7 Dual Formulation: Key Theorem

Theorem: Optimal weights for MSE + L_2 regularization: $w = \sum_{j=1}^n \alpha_j x^{(j)} = X^T \alpha$

where $\alpha \in \mathbb{R}^n$ (coefficient vector)

- Weights = linear combination of training vectors
- Number of parameters: n (training samples), not d (features)
 - $\Rightarrow$ works for high-dimensional/infinite feature spaces
- Foundation for kernel methods, SVM

4.8 Normal Equation (Ridge)

$$X^T (Xw - y) + n\lambda w = 0 \Rightarrow X^T (Xw - y) + n\lambda I w = 0$$

Closed-form: $w = (X^T X + n\lambda I)^{-1} X^T y$

Using Woodbury matrix identity:

$$w = X^T (X X^T + n\lambda I)^{-1} y = X^T \alpha$$

where $\alpha = (X X^T + n\lambda I)^{-1} y$

5 Neural Networks

5.1 Neuron Model

Basic neuron: input features $x_0, \dots, x_d$ with weights $w_0, \dots, w_d$ $z = \sum_{j=0}^d w_j x_j$ (aggregate)

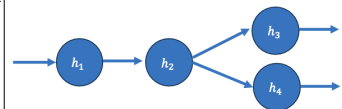
$a = g(z)$ (activate) output: $\hat{y} = a$

g : activation function (non-linear). w_0 (bias term, $x_0 = 1$).

5.2 Common Activation Functions

Sigmoid	$g(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$	$g'(z) = \sigma(z)(1 - \sigma(z))$
Identity	$g(z) = z$	$g'(z) = 1$
Tanh	$g(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$g'(z) = 1 - \tanh^2(z)$
ReLU	$g(z) = \max(0, z)$	$g'(z) = \begin{cases} 0, & z < 0 \\ 1, & z > 0 \end{cases}$
Leaky ReLU	$g(z) = \max(az, z) \ (a > 0)$	$g'(z) = \begin{cases} a, & z < 0 \\ 1, & z > 0 \end{cases}$

5.3 Neural Networks

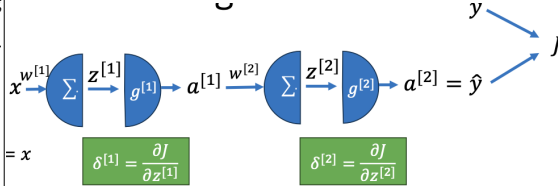


5.4 Feed-Forward Neural Networks (FFNNs)

Special case: computation graph is DAG (input $\rightarrow$ output, no cycles).

- Layer:** group of parallel neuron models $l_w: X_{l\text{in}} \rightarrow X_{l\text{out}}$
- Fully-connected (FC) layer:** all inputs connected to all neurons
- Depth:** number of sequential layers (excl. input)
- Width:** number of neurons per layer

5.5 FCNN: Matrix Representation



Single layer: $x \in \mathbb{R}^d \rightarrow \hat{y} = g(W^T x)$ where $W \in \mathbb{R}^{d \times m}$ (m outputs)

$\hat{y} = g(W^T x)$ **Multiple layers:** composition of matrix operations

$$\hat{y} = g^{[L]} \left(W^{[L]T} g^{[L-1]} \left(\dots g^{[1]} \left(W^{[1]T} x \right) \dots \right) \right)$$

Or Simply:

$$z^{[l]} = W^{[l]T} \cdot a^{[l-1]} + b^{[l]} \text{ and } a^{[l]} = g^{[l]} \cdot z^{[l]}$$

$b^{[l]}$: bias at layer l | $W^{[l]}$: Weights at layer l | $z^{[l]}$: pre-activation value at layer l | $a^{[l]}$: post-activation input at layer l , **input of layer $l+1$**

5.6 Gradient Formula

$$\frac{\partial \mathcal{L}}{\partial W^{[l]}} = \delta^{[l]} (a^{[l-1]})^T$$

5.7 Derivatives of Common Loss Functions

Mean Squared Error (MSE): $L = \frac{1}{2n} (y - \hat{y})^2$

$$\frac{\partial L}{\partial \hat{y}} = \frac{1}{n} \cdot (y - \hat{y}) \cdot (-1) = -\frac{1}{n} (y - \hat{y}) \text{ or } \frac{1}{n} (\hat{y} - y)$$

Loss range: $L \in [0, \infty)$ (always non-negative), Derivative range:

$$\frac{\partial L}{\partial \hat{y}} \in (-\infty, \infty) \text{ (any real number)}$$

Binary Cross-Entropy (BCE): $L = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$$

Loss range: $L \in (0, \infty)$ (approaches 0 as $\hat{y} \rightarrow y$, ∞ as $\hat{y} \rightarrow 1 - y$),

Derivative range: $\frac{\partial L}{\partial \hat{y}} \in (-\infty, -1) \cup (1, \infty)$ (negative when $y = 1$, positive when $y = 0$)

Absolute Error: $L = |y - \hat{y}|$

$$\frac{\partial L}{\partial \hat{y}} = -\text{sign}(y - \hat{y})$$

Loss range: $L \in [0, \infty)$ (non-negative), Derivative range: $\frac{\partial L}{\partial \hat{y}} \in \{-1, 0, 1\}$

(sign function; 0 at $y = \hat{y}$)

5.8 The Error Signal: $\delta^{[l]}$

$$\delta^{[l]} = -\frac{\partial \mathcal{L}}{\partial z^{[l]}}$$

- Gradient of loss w.r.t pre-activation at layer l .
- "how much each neuron at layer l contributed to the final error."
- Controls which direction and magnitude of change of weights & biases
- Loss function w.r.t update in weights at layer l**
- For each layer l , we look at:

$$\frac{\partial \mathcal{L}}{\partial W^{[l]}} = \delta^{[l]} (a^{[l-1]})^T$$

- Gradient magnitude depends on $\delta^{[l]}$.
- Backprop structure (key mechanism)**
-

$$\delta^{[l]} = (W^{[l+1]})^T \delta^{[l+1]} \odot \sigma'(z^{[l]})$$

- Unrolling this through N layers:

$$\delta^{[1]} = \left(\prod_{k=1}^{N-1} (W^{[k+1]})^T \right) \delta^{[N]} \odot \left(\prod_{k=1}^{N-1} \sigma'(z^{[k]}) \right)$$

- if N is large, gradient vanishes at early layers

5.9 Dying ReLU

1. Error signal formula: $\delta^{[l]} = (W^{[l+1]})^T \delta^{[l+1]} \odot g'(z^{[l]})$

2. For ReLU: $g'(z^{[l]}) = \begin{cases} 1 & z^{[l]} > 0 \\ 0 & z^{[l]} \leq 0 \end{cases}$

- If $z^{[l]} \leq 0$, then the error signal at that neuron becomes: $\delta^{[l]} = 0$
- That neuron does not contribute to backpropagation (no gradient flows through it).
- Leaky ReLU fixes dying ReLU by ensuring $g'(z) = \alpha > 0$ even when $z \leq 0$, so gradients do not become zero.
- This keeps $\delta^{[l]} \neq 0$, allowing weights to keep updating and preventing neurons from becoming permanently inactive.

5.10 Universal Approximation (Informal)

Single **hidden** layer FCNN with sufficient width + non-linear activation $\Rightarrow$ can approximate any continuous function on bounded domain.

- Sigmoid, Tanh, ReLU: all valid activations
- Why multiple layers then? Computational efficiency, expressiveness

5.11 Regression Tasks

Single output: $y \in \mathbb{R}$ (temperature prediction)

- Predictor: single neuron with **identity activation**
- Loss: $J_{\text{MSE}}(w) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2$

Multi-output: $y \in \mathbb{R}^m$ (temp + humidity)

- Predictor: m neurons with identity activation

Loss: $J_{\text{MSE}}(w) = \frac{1}{2n} \sum_{i=1}^n \sum_{j=1}^m (\hat{y}_j^{(i)} - y_j^{(i)})^2$

5.12

Classification

$y \in \{0, 1\}$ (rain or not)

- Predictor: single neuron with **sigmoid activation**
- $\hat{y} = \sigma(z) \in [0, 1]$ (prob. of class 1)
- Loss: $J_{\text{BCE}}(w) = \frac{1}{n} \sum_{i=1}^n \text{BCE}(y^{(i)}, \hat{y}^{(i)})$

5.13

Multi-class Classification

$y \in \{1, \dots, K\}$ (rain intensity: drizzle, light, heavy)

- Predictor: K neurons with **softmax activation**
- Softmax: $\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \in [0, 1]$ (prob. class i)

Loss: $J_{\text{CCE}}(w) = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^K y_j^{(i)} \log(\hat{y}_j^{(i)})$ (categorical cross-entropy)

5.14

One-Hot Encoding

Represent categorical labels as vectors: each class $\rightarrow$ vector with 1 in position of class, 0 elsewhere. Problem is high dimensionality and not useful representation of data.

Class	Vector
Drizzle	$[1, 0, 0]$
Light	$[0, 1, 0]$
Heavy	$[0, 0, 1]$

5.15

Multi-label Classification

$y \in \{0, 1\}^m$ (rain intensity AND cyclone risk independently)

- Predictor: m neurons with **sigmoid activation** (independent outputs)
- Loss: sum of binary cross-entropy for each label

5.16

Learning via Gradient Descent

Minimize loss $J(w)$ by iterative updates:

$$w \leftarrow w - \alpha \nabla_w J(w)$$

α : learning rate. $\nabla_w J(w)$: gradient computed via **backpropagation**.

5.17

Backpropagation

Algorithm: efficiently compute gradients $\frac{\partial J}{\partial w[l]}$ via chain rule.

- Forward pass: compute outputs layer-by-layer
- Backward pass: compute gradients from output to input
- Complexity: $O(n_{\text{params}})$ (same as forward pass)

5.18

Data Format

Dataset $D = \{(x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})\}$:

- Features: $x^{(i)} \in \mathbb{R}^d$ (real-valued)
- Targets: $y^{(i)}$ (task-dependent: scalar, vector, categorical, multi-label)

6

Convolutional Neural Networks (CNNs)

FCNN limitations with images:

- Parameter explosion: 600x600 RGB image $\rightarrow$ 1.08M parameters for single hidden layer
- Loss of spatial structure: treats pixels as independent features
- Solution: exploit local structure via convolution

6.1

Convolution Operation

Kernel/Filter: small matrix (e.g., 3x3) slides over input, computing element-wise products + sum.

- Input: $H \times W \times C$ (height, width, channels)
- Filter: $K \times K \times C$ (shared weights)
- Output: $(H - K + 1) \times (W - K + 1)$ per filter
- Stride**: step size of kernel movement (stride 2 $\implies$ half spatial dimension)
- Padding**: add zeros around input to preserve spatial dimension
- Formula for Output of CNN Layer** (input= $H \times W \times C$, Kernel= $h \times w \times c$, stride = s , padding = p)

$$H_{\text{out}} = \left\lfloor \frac{H - h + 2p}{s} \right\rfloor + 1, \quad W_{\text{out}} = \left\lfloor \frac{W - w + 2p}{s} \right\rfloor + 1, \quad C_{\text{out}} = n_{\text{filters}}$$

Output shape: $H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}}$

6.2

Convolutional Layer

m filters + bias $\rightarrow m$ output feature maps. Learnable parameters: $m \cdot (K \cdot K \cdot C + 1)$ (independent of input size).

- Activation**: ReLU applied per-element
- Receptive field**: region of input affecting output neuron (grows with network depth)
- Receptive field formula**: $r_i = r_{i-1} + (k_i - 1)j_{i-1}, \quad j_i = j_{i-1} \cdot s_i$
 k_i = kernel size at layer i
 s_i = stride at layer i ,
 j_i = jump at layer i ,

6.3

Pooling Layers

Reduce spatial dimensions, aggregate information:

- Max pooling**: $\max(2 \times 2)$ window $\rightarrow$ single value (translation invariance)
- Average pooling**: mean of window (smooth aggregation)
- No learnable parameters
- Stride typically matches pool size

6.4

CNN Architecture

Typical pattern: conv $\rightarrow$ ReLU $\rightarrow$ pool $\rightarrow$ repeat. Final layers: flatten + fully-connected for classification.

- Early layers: detect low-level features (edges, colors)
- Deep layers: detect high-level features (shapes, objects)
- Hierarchical feature extraction via receptive field growth

6.5

Sequential Data Representation

Types:

- Video: 4D tensor (T, H, W, C) (time, height, width, channels)
- Audio: 1D sequence (waveform samples)
- Text: sequence of words (variable length)

6.6

Recurrent Neural Networks (RNNs)

Process sequential data by maintaining hidden state:

$$h^{[t]} = \tanh(W_h h^{[t-1]} + W_x x^{[t]} + b)$$
$$\hat{y}^{[t]} = W_y h^{[t]} + b_y$$

- $h^{[t]}$: hidden state at time t
- $x^{[t]}$: input at time t
- Shared parameters (W_h, W_x, W_y) across timesteps

6.7

RNN Variants

- Bidirectional RNN**: process sequence forward + backward, concatenate $h_{\text{fwd}}^{[t]}$ and $h_{\text{bwd}}^{[t]}$ for output
- Stacked RNN**: multiple RNN layers (output $h^{[l]}$ of layer $l \rightarrow$ input to layer $l + 1$)
- Cardinality**: one-to-many, many-to-one, many-to-many: s -to- o where s is number of steps and o is number of output. note that output can also be a sequence

6.8

RNN Bottlenecks

- Sequential computation**: cannot parallelize over timesteps $\implies$ slow training/inference
- Memory decay**: hidden state weakly depends on distant past (vanishing/exploding gradients during backprop)
- Long-term dependencies: hard to capture with tanh activation

6.9

Text Representation

One-hot encoding: vocabulary size $V \rightarrow$ each word = vector with 1 in position, 0 elsewhere (V -dimensional).

- Issue: sparse, no semantic similarity captured
- Solution: **embedding layer** learns dense representation

6.10

Embedding Layer

Learnable lookup table $E \in \mathbb{R}^{V \times d}$ (V words, d dimensions):

- Input: word index i
- Output: $E[i, :] \in \mathbb{R}^d$ (dense vector)
- Learns semantic similarity: similar words $\rightarrow$ nearby embeddings
- Parameters: $V \cdot d$ (learned via gradient descent)

6.11

Attention Mechanism

Problem: RNN hidden state becomes bottleneck. Solution: allow model to look at **all** previous timesteps, not just last hidden state.

- Query vector: "what am I looking for?"
- Key vector: "what information do I have?"
- Value vector: "what should I extract?"
- Relevance: $\text{score}(q, k) = q^T k$ (dot product)

6.12

Self-Attention: Query, Key, Value

For timestep t , transform hidden state $h^{[t]}$ via learnable matrices:

$$q^{[t]} = W_q h^{[t]}, \quad k^{[t]} = W_k h^{[t]}, \quad v^{[t]} = W_v h^{[t]}$$

Compute relevance of all timesteps to current query:

scores = $QK^T \in \mathbb{R}^{T \times T}$ (where $Q = [q^{[1]}, \dots, q^{[T]}], K = [k^{[1]}, \dots, k^{[T]}]$)

Apply softmax + weighted sum of values:

$$\text{attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- $\frac{1}{\sqrt{d_k}}$ scaling: prevent dot products from exploding
- Output: context vector for each timestep (attended information from all timesteps)

6.13

Self-Attention Advantages

- Parallel computation: all timesteps processed simultaneously (no sequential loop)
- Long-term dependencies: direct connections between all pairs of timesteps (no memory decay)
- Interpretability: attention weights show which words attended to

7

Unsupervised Learning with NN

7.1

Supervised Learning Bottleneck

- Supervision Signal**: Labels $y^{(i)}$ explicitly compared $\rightarrow$ precise error correction
- Annotation Cost**: Time + Money + Domain expertise (e.g., doctors labeling medical scans)
- Reality**: Vast majority of data (text, images, video) $\rightarrow$ no human annotations
- Challenge**: How to leverage unlabeled data?

7.2

Unsupervised Learning Fundamentals

Definition: Train on only inputs $\{x^{(i)}\}$, no human labels. Generate supervision from data itself.

Three-Stage Framework:

- Pretext Task**: Auxiliary task with auto-generated pseudo-labels
- Self-Supervised Pre-Training**: Model solves pretext task on unlabeled data
- Supervised Fine-Tuning**: Transfer learned features to downstream task

7.3

Self-Supervised Learning Framework

Design New Task (Pretext Task):

- Formulate task: targets (pseudo-labels) auto-generated from unlabeled input
- Generate supervision directly from data itself, without human annotation
- Task Requirement: must force model to capture useful features & underlying structures
- Train model to solve new task $\rightarrow$ model supervises itself

7.4

Rotation Prediction Pretext Task

Setup: Given unlabeled images $\{x^{(i)}\}$

Transformation: For each image $x^{(i)}$, generate 4 rotated versions:

- $j = 0$: rotate $0 \rightarrow x^{(i)[0]} \dots$ Total rotated images: $4n_1$

Pseudo-label Generation: One-hot encode rotation angle $y^{(i)[j]} \in \mathbb{R}^4$

- $j = 0$: $y^{(i)[0]} = [1, 0, 0, 0]^T \dots$ 4 classes defined by rotation angle

Architecture: Feature Extractor + Predictor

- Feature Extractor**: Conv layer $\rightarrow$ Pooling layer $\rightarrow$ Conv layer $\rightarrow$ Pooling layer $\rightarrow$ Flatten
- Predictor**: FC layers + 4 outputs $\hat{y}_1, \hat{y}_2, \hat{y}_3, \hat{y}_4$
- Loss**: Cross-Entropy between $\hat{y}^{(i)[j]}$ and $y^{(i)[j]}$

Why Rotation? Understanding image content requires:

- Object orientation: recognizing what is "upright"
- Shape & structure: edges, contours, pose

- $\Rightarrow$ Learned features useful for downstream tasks like image classification

7.5

Transfer Learning & Fine-Tuning

Downstream Task: Binary image classification (e.g., bird vs. non-bird)

Architecture:

- Feature Extractor**: Reuse from rotation model, initialize with pretrained weights
- New Predictor**: Task-specific FC layer (1 output for binary), initialize with **random weights**

Fine-Tuning Strategies:

- Update every single weight in entire model
- Update subset of model weights (e.g., only predictor, or only later layers)

Loss: Binary Cross-Entropy $\text{BCE}(\hat{y}^{(i)}, y^{(i)})$ where $y^{(i)} \in \{0, 1\}$

8

Uncategorized

8.1

Linear & Logistic Regression Cheatsheet

8.1.1

Linear Regression Basics

Model: $h_w(x) = w_0 + w_1 x_1$ (bias w_0 , or use dummy $x_0 = 1$).

MSE Loss: $L_{\text{MSE}}(w) = \frac{1}{2n} \sum_{i=1}^n (h_w(x^{(i)}) - y^{(i)})^2$

Example (6A): $w = [2, 1]^T$, data: $(0, 0), (2, 1), (3, 2)$

$h(x^{(1)}) = 2 + 0 = 2$, $h(x^{(2)}) = 2 + 2 = 4$, $h(x^{(3)}) = 2 + 3 = 5$

$L_{\text{MSE}} = \frac{1}{6} [2^2 + 3^2 + 3^2] = \frac{22}{6} = \frac{11}{3}$

8.1.2

SGD Update (6B)

Pick random point i . Update:

$$w \leftarrow w - \alpha \cdot (h_w(x^{(i)}) - y^{(i)}) \begin{bmatrix} 1 \\ x^{(i)} \end{bmatrix}$$

Example: Random point $i = 2$, $\alpha = 1/3$.

Residual = $h(x^{(2)}) - y^{(2)} = 4 - 1 = 3$

$$w' = \begin{bmatrix} 2 \\ 1 \end{bmatrix} - \frac{1}{3}(3) \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

8.1.3

Linear Dependence & Convexity (6C)

If features are linearly dependent (e.g. $x_2 = 3x_1$): loss is convex (not strictly), so GD may converge to *different weights* but *same loss*. Independent features $\implies$ strictly convex $\implies$ unique weights. **Key**: same loss, possibly different weights. $A : x \rightarrow h^A, x' \rightarrow h^{A'}$ $B : x \rightarrow h^B, x' \rightarrow h^{B'}$ training on x (linearly dependent) may produce different weights but same loss. training on x' (linearly independent) will produce same weights same loss

8.1.4

Feature Scaling for Stability (6D)

When features have very different scales (e.g. $x_3 \sim 1000$ vs others ~ 1):

- Standardization**: $(x - \mu)/\sigma$ $\checkmark$
- Min-max scaling**: $(x - x_{\min})/(x_{\max} - x_{\min})$ $\checkmark$
- Normal equation fails if features are linearly dependent (non-invertible $X^T X$)

8.1.5

Logistic Regression (6E)

Use $\sigma(z) = 1/(1 + e^{-z})$ when predicting *probability*. Need bias w_0 if $P > 0.5$ when all $x_i = 0$ (since $\sigma(0) = 0.5$, need $w_0 > 0$). Write as $P(Y|X) = \sigma(w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3)$. Note: $P(Y|X)$ not $P(Y|w)$.

8.1.6

Normal Equation (6F)

$w = (X^T X)^{-1} X^T y$

Diagonal matrix inverse: flip each diagonal entry $d_i \rightarrow 1/d_i$.

Example: $X = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, y = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$

$$X^T X = \begin{bmatrix} 1 & 0 \\ 0 & 4 \end{bmatrix}, (X^T X)^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 1/4 \end{bmatrix}$$
$$X^T y = \begin{bmatrix} 5 \\ 12 \end{bmatrix}, w = \begin{bmatrix} 1 & 0 \\ 0 & 1/4 \end{bmatrix} \begin{bmatrix} 5 \\ 12 \end{bmatrix} = \begin{bmatrix} 5 \\ 3 \end{bmatrix}$$

8.1.7

Inverting 2x2 Matrix

For $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$, $A^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$

8.1.8

distance of point x from some hyperplane P (or B for boundary)

- $d = \frac{w^T x + b}{\|w\|}$, OR
- find point z on P ($\frac{x}{y} \cdot w_1 w_2 + b = 0$)
 - get $\vec{u} = \vec{x} - \vec{z}$
 - find $\text{Proj}_w \vec{u} = \frac{\vec{u} \cdot \vec{w}}{\|\vec{w}\|}$